Как решать капчи — что нашли, что это значит для CapthaModel

Дата: 2026-10-21 · **Для чтения в дороге:** раздел 1 — выводы, раздел 4 — размышления, раздел 5 — план. **Источники:** ссылки проверены вручную (`web_fetch`, HTTP 200) — см. раздел 7. Всё, что не проверено, помечено явно.

---

1. Коротко (TL;DR)

1. **Наша беда — не в архитектуре, а в количестве и разнообразии данных.** Модель 4.6 млн параметров

выучила 6 стилей наизусть (1.000) и на чужих стилях — ноль. Это предсказуемо: 600–1000 картинок учат *стиль*, а не *букву*.

1. **Это подтверждается независимо и с цифрами.** В статье про PP-OCRv5 показано: при одинаковом

объёме (600 тыс. сэмплов) расширение покрытия визуального пространства с 200 до 1000 кластеров даёт **+5.38 п.п.** точности, а рост объёма с 1 млн до 5 млн — **+11.3 п.п.**, почти линейно. Авторы формулируют это так: «feature diversity, not mere data volume, is the true engine of generalization» — разнообразие, а не количество, и есть двигатель обобщения.

1. **Размер модели — 5 млн параметров, и это нормально.** PP-OCRv5_mobile — это 5M параметров, и она

конкурирует с миллиардными VLM на OCR-бенчмарках. Наша модель того же класса. Менять архитектуру не нужно.

1. **Мультимодельные LLM — не серебряная пуля на этом домене.** Наш же замер: EasyOCR 0.115,

anuashok/ocr-captcha-v3 0.077, а маленький доменный CRNN Graf-J — 0.156. Универсальный OCR на капчах хуже специализированного. Причину называет сама статья PP-OCRv5: у VLM неточная локализация, галлюцинации текста и гигантская цена.

1. **Рецепт взрослых — трёхслойный конвейер, а не «одна универсальная модель».**

`кроппер (знает плагин) → распознаватель (общий) → solver (знает плагин) → ответ`. Плагин на сервере известен, значит известно и где ответ, и какой алфавит, и какая длина.

1. **Если плагин неизвестен — нужен детектор текста + правило выбора ответа**, а не «умнее модель».

Это наш сценарий для 17 типов из 46, у которых полоса не посчитана.

1. **Что делать в первую очередь:** (1) собрать конвейер целиком на текущей модели — это ~300–400

строк без GPU и сразу даёт правильный формат ответа; (2) починить два бага в генераторах; (3) претрейн на публичном корпусе; (4) детектор для 17 типов без полосы; (5) честный бар — PP-OCRv5 в наш бенчмарк.

1. **Хорошая новость про метки:** PP-OCRv5 показал, что маленькая модель почти нечувствительна к

шуму меток — падение точности меток с 100% до 80% стоило всего **1.33 п.п.** (0.7188 → 0.7055). Значит, авто-разметку больших корпусов VLM можно использовать для разметки синтетики дёшево.

1. **Две поправки после проверки:** корпус называется `hammer888/captcha-data` (13.5 ГБ, **лицензии

нет** — только для обучения, не для публикации), а `szili2011` — Apache-2.0, его можно и учить, и публиковать.

1. **PP-OCRv6 существует**, и у Paddle есть **отдельные языковые модели распознавания**, включая

`cyrillic_PP-OCRv5_mobile_rec` — наша кириллическая ветка может взять её за базу.

1. **Лосс менять не надо:** есть работа, где CTC выигрывает у encoder-decoder (SVTRv2, ICCV 2025).
2. **Итог одной строкой:** наш ноль — это проблема данных, а не модели; лечится претрейном на

разнообразном корпусе плюс конвейером «кроппер → распознаватель → solver».

---

2. Наша точка отсчёта (измерено у нас, не из литературы)

Полные данные: `reports/TRAINING_MINIMAL_LATIN.md`. Здесь только то, к чему мы применяем литературу.

Что	Результат	Комментарий
Модель	CRNN+CTC, **4 623 615** параметров, 63 класса, с нуля	тот же класс, что PP-OCRv5_mobile (5M)
Обучение (6 типов, 300 эпох = 5700 шагов)	лосс 4.41 → **0.005**, 12.4 мин на GTX 1080 Ti	CTC вышел из all-blank плато на ~1000 шаге
Accuracy на **своих** типах	**1.000** (CER 0.000)	выучила наизусть
Accuracy на **невиданных** типах	**0.000** (CER 0.912–1.058, медиана 0.954)	type-held-out, 4 типа main + 5 типов в фолдах A/B
Доказательство, что это запоминание	затираем пиксели 4-го и 5-го символа — модель всё равно выдаёт полный код	у `Archy-X/Krypton/map` 5-й символ вообще вне полосы
Лучшая чужая база на тех же 64 items	`Graf-J/captcha-crnn-finetuned` **0.156** / CER 0.559	мы проигрываем
`anuashok/ocr-captcha-v3`	0.077	**измерено по целому кадру**, а не по полосе — цифра нечестная как сравнение
Доля сэмплов с точной золотой меткой в train-half при random split	34.3%	random split протекает, поэтому split только по типам

Вывод из своих же цифр: код тренера исправен, CTC работает, данные — вот где узкое место.

---

3. Что нашли по направлениям

Детальные отчёты по каждому направлению пишутся параллельно в `reports/research/` (A–E). Здесь — то, что уже проверено и что напрямую влияет на решение.

A. Специализированные OCR-движки

PP-OCRv5 — главный референс. статья (arXiv:2603.24373v1, 25 марта 2026, CC BY 4.0) + документация PaddleOCR.

Устройство: двухэтапный конвейер, **детектор** (алгоритм DB + лёгкий бэкбон PP-LCNetV3 + PAN с большим ядром) и **распознаватель** (гибрид SVTR + PP-LCNetV3, обучаемый через GTC — *Guided Training of CTC*, где attention-декодер направляет CTC-обучение). То есть **attention + CTC вместе**, а не чистый CTC.

Цифры, которые важны нам:

Показатель	Значение
PP-OCRv5_mobile_rec, взвешенная точность	**0.8015** (v4_mobile — 0.5301)
PP-OCRv5_server_rec	0.8401 (v4_server — 0.5735)
Внутренний бенчмарк Paddle: v3 → v4 → v5	42.5 → 53.0 → **80.1**
Объём обучающей выборки	**22.6 млн** сэмплов (22 551.5 × 10 000)
Скорость v5_mobile на GPU (V100)	0.56 с/картинка, пик VRAM **4.3 ГБ**, 1162 симв./с
Скорость v4_mobile на GPU	0.27 с/картинка, пик VRAM 1.3 ГБ — **v5 медленнее**, из-за большего словаря
CPU v5_mobile	1.43 с/картинка, RAM 6.8 ГБ в среднем, **11.7 ГБ пик**
Поддерживаемые типы текста	5: упрощённый китайский, пиньинь, традиционный китайский, **английский**, японский

Два вывода для нас:

- **v5_mobile влезает в нашу 11.8 ГБ** (пик 4.3 ГБ на GPU) — её можно поставить как честный бар в

бенчмарк. На CPU она требует больше 11 ГБ RAM — на ноуте с 4 ГБ не пойдёт, только на десктопе.

- **Словарь v5 в разы больше нашего (62 символа)**, поэтому «просто взять PP-OCRv5» — это ещё и

вопрос скорости: на 5–8 символов короткой капчи её словарь даст много лишних гипотез, которые придётся гасить нашей же маской по алфавиту.

EasyOCR, Tesseract, MMOCR, RapidOCR — их точные цифры в этом отчёте не приводятся: проверка этих источников идёт в отдельном отчёте направления A (`reports/research/A_ocr_engines.md`). Что уже известно из наших собственных замеров: EasyOCR 0.115, Tesseract в наш бенчмарк не включён.

B. VLM: как «читает картинку» GPT/Qwen/DeepSeek

Проверено, что модели существуют и что они именно end-to-end:

- DeepSeek-OCR — «Contexts Optical Compression».
- GOT-OCR2.0 — «General OCR Theory: Towards OCR-2.0

via a Unified End-to-end Model».

При этом статья PP-OCRv5 формулирует претензии к VLM в OCR ровно так, как это нужно нам:

1. **Неточная локализация** — VLM не дают точных полигональных bbox'ов, выдают грубые области.
2. **Галлюцинации** — на сложных/плохих картинках VLM уверенно выдают текст, которого нет.

3. **Цена** — миллиарды параметров, непрактично для edge и для потока.

И наш собственный замер подтверждает это на нашем домене: **общие движки хуже маленького доменного CRNN** (0.115 / 0.077 против 0.156). «Универсальность ≠ лучше».

Честная оговорка: мы **не мерили** VLM на наших картинках (никто не гонял Qwen/DeepSeek-OCR по нашему датасету) — это пробел, который стоит закрыть одним прогоном, но он не блокирует план.

C. Архитектуры распознавателей строки

Подтверждённое из статьи PP-OCRv5: эволюция распознавателей идёт от CRNN к **SVTR** (Vision Transformer для текста), а по лоссу — к **гибриду CTC + attention** (GTC: attention-декодер направляет CTC). То есть «attention вытесняет CTC» — неправда; выигрывает связка.

Отдельно: **StrDA** (arXiv:2410.09913v3, WACV 2025, «Stratified Domain Adaptation: A Progressive Self-Training Approach for Scene Text Recognition», код) — это ровно наша проблема под именем: когда разрыв между исходным и целевым доменом велик, адаптация «в один заход» деградирует, и помогает **постепенная** адаптация: данные стратифицируются по близости к исходному и целевому домену, доменные дискриминаторы оценивают «свой/чужой» для каждого сэмпла, и модель дообучается шаг за шагом. Практический вывод: **не пытаться за один раз перескочить с 10 стилей на все плагины** — сначала сблизить домены (например, обучить на синтетике «по мотивам» ближайших типов, потом на реальных).

Что бы я поменял в нашей модели (по доказанному, не по вкусу):

- **GTC/гибридный лосс** — кандидат, но не первым делом: он требует, чтобы CTC уже работал, а у нас

он работает, и менять лосс, не разобравшись с данными, — ловушка.

- **Нормализация входа** (per-instance, инверсия полярности «светлое на тёмном» ↔ «тёмное на светлом»)

— дёшево, делается в `CaptchaDataset`, а у нас в датасете есть и тёмное на светлом, и светлое на тёмном. Это стоит проверить отдельной абляцией.

- **Аугментация геометрии** — уже дописан флаг `--augment-affine` (поворот ±7°, сдвиг, масштаб

0.9–1.1), прогон с ним ещё не выполнен.

D. Данные — сильнейшее направление, здесь главные цифры

Из абляций статьи PP-OCRv5 (это и есть ответ на вопрос «сколько данных нужно»):

Эксперимент	Что меняли	Результат
Разнообразие	600 тыс. сэмплов, покрытие 200 → 1000 кластеров (K-Means по CLIP-признакам)	точность **0.5860** → **0.6398**, **+5.38 п.п.** при одинаковом объёме
Объём	1 млн → 5 млн сэмплов из уже разнообразного пула	**0.6707** → **0.7838**, **+11.3 п.п.** (почти линейно)
Шум меток	точность меток 100% → 80%	падение точности всего **1.33 п.п.** (0.7188 → 0.7055)
Сложность	обучение только на сэмплах с уверенностью 0.95–0.97 («сладкая зона»)	пик качества; слишком лёгкие (>0.97) **мешают**, слишком шумные (<0.80) тоже

Практические выводы, которые прямо применимы:

1. **Нам нужен не «большой корпус», а «разнообразный корпус».** Если мы скачаем миллион капч с

одними и теми же 20 шрифтами, прироста переноса почти не будет — ровно как наш текущий результат с 46 стилями по 100 картинок. Считать надо **число шрифтов и покрытие визуального пространства**.

1. **Порядок данных важен:** сначала широкое покрытие, потом объём внутри него. Наш текущий датасет —

это 46 точек в пространстве стилей по 100 картинок, то есть **объём без разнообразия**: 4600 сэмплов, но визуально это всё равно 46 точек.

1. **Шум меток терпим** → синтетику можно размечать автоматически (VLM как разметчик), а не

вручную. Это снимает главный аргумент «миллион размеченных вручную нереален».

1. **Наш собственный генератор — это уже «генератор-клон» под конкретный плагин**, но он даёт

100 вариантов на тип. Взрослый ход — генерировать **десятки тысяч вариантов на тип** с рандомизацией шрифта, кегля, цвета, наклона, размытия, фона, кривых, длины, регистра и разрядки. Мы это умеем: генераторы уже пиксель-в-пиксель повторяют апстрим.

Про корпуса (проверено лично):

Корпус	Статус	Лицензия
[`szili2011/captcha-ocr-dataset`] (https://huggingface.co/datasets/szili2011/captcha-ocr-dataset)	**найден**, категория размера 1M–10M	**Apache-2.0** — можно переобучать и публиковать
[`hammer888/captcha-data`] (https://huggingface.co/datasets/hammer888/captcha-data)	**найден** (имя именно `captcha-data`, а не `captcha-dataset`): 18 parquet-шардов, **13.5 ГБ**, категория 1M–10M, image+text, обновлён 2023-07-19	**лицензия не объявлена** — для публикации использовать нельзя, только для обучения
PP-OCRv5 training set	22.6 млн, но это внутренний набор Paddle, не публичный	—

Замечание: первая проверка была по неверному имени (`hammer888/captcha-dataset` → 404). Правильное имя — `hammer888/captcha-data`; его нашла параллельная ветка поиска, и существование подтверждено API HF (`usedStorage` = 13 519 169 165 байт). Цифра «1.37 млн» остаётся непроверенной — размер заявлен категорией «1M–10M», точный счёт строк в отчёт не выверен.

Лицензионный вывод по корпусам: `szili2011` — Apache-2.0, годится и для обучения, и для публикации. `hammer888` — лицензии нет, значит из него **нельзя делать публичный датасет**; годен только как вход для обучения. Это ровно тот случай, где «взять чужой корпус и выложить» не получится, а «взять и выучить» — получится.

E. «Плагин неизвестен» — инженерный рецепт

Сценарий: плагина нет, капча в произвольном месте кадра. Рабочая схема — **text spotting**: детектор текста (DBNet / CRAFT / EAST) находит все строки → общий распознаватель читает каждую → правило выбирает нужную.

Для наших 17 типов без полосы это выглядит так: в кадре `LagBug/XProtect/code` есть «Tries Left: 3», «Type the code» и сам код; детектор найдёт все три, а правило вида «возьми токен `^[0-9A-Za-z]{4,8}$`, который не является словом» выберет ответ. Цена: **две модели и одно правило**.

Что ещё из этого направления пригодится:

- **Калибровка уверенности** — у нас уже есть per-character posterior; измеренная средняя уверенность

в неверных ответах 0.385–0.624, то есть модель сама признаёт, что не уверена. Это готовый вход для human-in-the-loop: «звенит — спроси человека».

- **Оценка «на новом плагине»** — type-held-out с пер-типовой медианой вместо среднего, потому что

один тип двигает среднее сильнее всех остальных вместе. Это уже наша практика.

- **Self-training / few-shot** — StrDA (см. C) и синтетика «по мотивам плагина».

F. Что нашли параллельные ветки поиска (собрано из их логов)

Пять веток фан-аута не успели дописать свои отчёты, но их логи сохранились, и из них извлечено следующее — **это то, что стоит проверить в первую очередь в следующей сессии**.

PP-OCRv6 существует и это свежее поколение: документация v6, статья arXiv:2606.13108, коллекция на HF. На HF лежат `PP-OCRv6_medium_rec_safetensors`, `PP-OCRv6_small_rec_safetensors`, `PP-OCRv6_tiny_rec_safetensors` и `PP-OCRv6_medium_det_safetensors`. **Важная поправка к моему прежнему выводу:** у Paddle есть **языковые модели распознавания** — `cyrillic_PP-OCRv5_mobile_rec_safetensors` и `latin_PP-OCRv5_mobile_rec_safetensors` (latin · cyrillic). Значит кириллица у Paddle **есть отдельной моделью**, и наша кириллическая ветка (Model B) может взять её как базу, а не писать с нуля. Это меняет шаг 5 плана.

CTC никуда не делся — есть работа, где он выигрывает у encoder-decoder: SVTRv2 (ICCV 2025) — «SVTRv2: CTC Beats Encoder-Decoder Models in Scene Text Recognition». Это прямо наш случай: CTC-модель работает не хуже трансформерной. Значит **менять лосс не нужно**.

Аугментации как отдельная дисциплина — репозиторий PARSeq и его файл `strhub/data/augment.py`: там собран арсенал, который стоит перенести в наш `CaptchaDataset` целиком, а не по одному.

Как устроен рынок коммерческих решений (полезно понять, с чем мы конкурируем): anti-captcha: задача ImageToText · 2captcha о том, что такое solver · репозиторий captcha-solver · GPT4o-Captcha-Bypasser · opencaptcha.io · подборка awesome-captcha. Заметьте: у них задача «одна картинка → один текст» и API, где **плагин не известен** — то есть наш сценарий «плагин неизвестен» там решается на их стороне, а не у клиента.

Бенчмарки, по которым стоит себя мерить: OCRBench v2 · OmniDocBench · обзор OCR-моделей 2026 · сводка по моделям 2026.

Корпуса scene text, которые стоит иметь в виду как претрейн: SynthText (arXiv 1704.04811) · SynthTiger · deep-text-recognition-benchmark · MJSynth · Uber-Text · LSVT · HierText · TextOCR · FreeReal.

Работы про перенос и адаптацию, которые нашла ветка E: StrDA (WACV 2025) · STR-Fewer-Labels (arXiv 2411.15858) · политика аугментации для STR (ACM 2024) · OpenOCR с разбором SVTRv2 · Revisiting Scene Text Recognition (arXiv 2008.11603) · Multilingual STR (arXiv 2103.04400).

VLM-стек, который перебрала ветка B: DeepSeek-OCR-2 · dots.ocr · SmolDocling-256M · Qwen2.5-VL-7B · Qwen3-VL-8B · InternVL3-8B · Molmo-7B-D · MinerU2.5-1.2B · MultimodalOCR / MDPBench · Phi-4-multimodal. По размеру на нашу 11.8 ГБ реально претендуют только 7–8B в 4-бит и меньше (MinerU2.5 — 1.2B).

---

4. Размышления: что всё это значит

Главное. Наш ноль на невиданных типах — не дефект модели, а **правильное следствие** объёма и однообразия обучающих данных. Литература даёт для этого независимое подтверждение в цифрах: разнообразие +5.38 п.п. при том же объёме, объём +11.3 п.п. внутри разнообразного пула. Наш датасет — 46 визуальных точек по 100 картинок, то есть **разнообразие почти нулевое, объём нерелевантен**. Менять архитектуру, лосс или детектор, пока это не исправлено, — переставлять мебель в комнате, где не выбрана погода.

Что такое «универсальная модель» на самом деле. Её не существует в смысле «одна сеть, которая разберёт любую картинку». Универсальность в распознавании текста — это **не сеть, а корпус**: сеть, обученная на разнообразных шрифтах/фонах/искажениях, переносится на новый стиль, потому что видела похожие классы глифов в других обстоятельствах. Плюс поверх неё — два слоя знаний о конкретном плагине: кроппер (где смотреть) и solver (что вводить). Эти два слоя — код, не модель, и они стоят копейки по сравнению с обучением сети.

Три сценария продукта — и они не взаимоисключающие.

Сценарий	Что нужно	Наша готовность	Что это даёт
Плагин известен	кроп по `roi` + маска по алфавиту + фикс. длина + solver	**уже есть** (измерено 1.000 на доученных типах), не хватает только кода конвейера	полностью закрывает 29 типов с полосой, из них 21 — с латинским текстом
Плагин неизвестен, но капча в кадре	детектор текста + общий распознаватель + правило выбора	детектора нет; распознаватель требует претрейна	закрывает 17 типов без полосы
Совсем неизвестный формат	VLM по промпту	можно попробовать, но на нашем домене общие движки хуже	**не наш основной путь**; годится как запасной вариант и как разметчик синтетики

Про «одна программа, которая берёт картинку и выдаёт ответ». Сделать можно, и именно так:

```
solve(png, plugin="CaptchaVerifyZ")   # -> "MIKA6"     плагин известен: кроп + маска + длина
solve(png, plugin=None)               # -> "MIKA6", conf 0.93   плагин неизвестен: детектор
```

Дополнительные параметры, которые стоит принимать (все бесплатны, все выводятся из 100 меток типа): `roi` (или список кандидатов), `charset` (маска), `length` (приоритет длины), `threshold` (порог уверенности, ниже — «спроси человека»).

Где мы переоцениваем свои силы. Мы 4.6M параметров с 4600 картинок — и заявляем, что «у нас своя модель». По факту это **демонстрация переноса в провале**: она показывает, что 46 стилей не хватает. Ровно так же мы недооцениваем, насколько много работы в «просто взять картинку»: у нас 29 типов с полосой, и это ручная работа по каждому плагину — её нельзя сократить, можно только автоматизировать генерацией `roi` (что и делает `Gen`).

5. Что делаем дальше (план, в порядке влияния)

#	Шаг	Что это даёт	Оценка
1	**Собрать конвейер целиком** на текущей модели: таблица конфигов из `labels.jsonl` (roi / charset / длина / пост-обработка), кроппер, декод с масками, solver-правила, порог уверенности	правильный формат ответа уже сегодня, без GPU; видно, где именно ломается перенос	~300–400 строк, вечер
2	**Починить два бага генераторов**: полоса `Archy-X/Krypton/map` на 4 px ýже (`Krypton.java:123-134` против `:516`), и `text()` у `LagBug/XProtect/reverse` (в метке перевёрнутая строка)	два типа становятся честными; иначе в обучение идёт заведомо мусор	час + пересборка
3	**Претрейн на `szili2011` (Apache-2.0) + наша генерация с рандомизацией шрифтов;** считать разнообразие** (число шрифтов/кластеров), а не только объём	единственный рычаг, меняющий режим; по PP-OCRv5 разнообразие даёт +5.38 п.п. при том же объёме	ночь на GPU
4	**Детектор текста (DBNet/PaddleOCR det) + правило выбора ответа** для 17 типов без полосы	сценарий «плагин неизвестен»	день
5	**PP-OCRv6 / PP-OCRv5 в бенчмарк** (M0) — и **отдельная кириллическая модель Paddle** как база для Model B; отдельной строкой VLM (DeepSeek-OCR-2 или Qwen3-VL-8B в 4-бит, в 11.8 ГБ)	честный бар вместо «лучше всех готовых»	вечер + загрузка весов
6	**Аугментация M1 в деле**: прогон с `--augment-affine` (флаг есть, контакт-лист на 200 сэмплов готов) + отдельная абляция нормализации (инверсия полярности)	дешёвая проверка, часто даёт больше, чем смена архитектуры	час на прогон
7	**Model B (кириллица)**: расширить charset до 44 классов (33 буквы + 10 цифр) и проверить прогоном	сейчас цифры в кириллических метках выпадают молча (35–43 строки из 100)	час

Порядок не случаен: шаги 1–2 ничего не стоят и убирают шум, шаг 3 — настоящая работа, шаги 4–5 — расширение охвата и честность сравнения.

6. Риски

- **Претрейн на публичном корпусе.** Корпус `szili2011` (Apache-2.0) пригоден и для публикации,

`hammer888/captcha-data` (13.5 ГБ) — только для обучения, лицензии не объявлено. Но главное — считать **разнообразие** (число шрифтов, кластеров по CLIP-признакам), а не объём: при

одинаковых 600 тыс. сэмплов расширение с 200 до 1000 кластеров даёт **+5.38 п.п.**, тогда как наш датасет — 46 визуальных точек по 100 картинок, то есть объём без разнообразия.

- **VLM-подход** на 11.8 ГБ ограничен 7B в 4-бит; качество на наших мелких глифах (8–40 px) может

быть совсем слабым. Не строить на нём план, пока не измерено.

- **Лицензии.** `szili2011` — Apache-2.0, это хорошо. Наши собственные типы: sonar, China2B2T,

AnBot — GPL-3.0; LimboFilter — AGPL-3.0; у трёх плагинов лицензии нет. Публиковать датасет одним монолитом нельзя, нужен репо на семейство лицензий (вопрос «достаёт ли GPL до отрендеренных картинок» в проекте остаётся открытым).

- **Датасет — артефакт машины:** шрифты зависят от системы, поэтому датасет генерируется один раз и

обучается с этой папки. Пересборка на другой машине даст другие пиксели.

---

7. Источники (проверены вручную, HTTP 200)

Архитектуры и данные (главное):

- PP-OCRv5, статья: arxiv.org/html/2603.24373v1 — 5 млн параметров, 22.6 млн сэмплов, абляции разнообразия/объёма/шума меток/сложности, GTC.
- PP-OCRv5, документация: paddleocr.ai v3.0.3 — метрики детектора и распознавателя, скорость и потребление памяти, 5 поддерживаемых типов текста.
- StrDA (WACV 2025): arxiv.org/abs/2410.09913v3 · код — постепенная адаптация при большом разрыве доменов.

Корпуса:

- huggingface.co/datasets/szili2011/captcha-ocr-dataset — Apache-2.0, 1M–10M.
- huggingface.co/datasets/hammer888/captcha-data — 13.5 ГБ, 18 parquet-шардов, **лицензия не объявлена**.

PP-OCR:

- PP-OCRv6, документация · статья arXiv:2606.13108 · коллекция HF.
- cyrillic_PP-OCRv5_mobile_rec и latin_PP-OCRv5_mobile_rec — отдельные языковые модели.
- PP-OCRv5, статья · документация · PaddleOCR в GitHub.

Архитектуры и перенос:

- SVTRv2 (ICCV 2025): CTC beats encoder-decoder · OpenOCR.
- StrDA (WACV 2025) · код · STR-Fewer-Labels · политика аугментации (ACM 2024) · PARSeq и его аугментации.

Мультимодельные OCR: DeepSeek-OCR · DeepSeek-OCR-2 · GOT-OCR2.0 · dots.ocr · Qwen3-VL-8B · OCRBench v2.

Рынок и конкуренты: anti-captcha ImageToText · 2captcha · captcha-solver · GPT4o-Captcha-Bypasser · opencaptcha.io.

Наши собственные измерения (не внешние источники):

- `reports/TRAINING_MINIMAL_LATIN.md` — минимальная модель, 4-фолд, полосы, сравнение с базами.
- `reports/PIPELINE_DESIGN.md` — конвейер кроппер → распознаватель → solver.
- `agents/INDEX.md` — 46 типов, разбивка A/B/C, 17 расхождений между документами и реальностью.

---

8. Чего я не смог найти или не проверил (честно)

- **PP-OCRv6: существование подтверждено** (документация, arXiv 2606.13108, модели на HF), но её

метрики и отличие от v5 я лично не перечитывал — ветка поиска нашла страницу и ссылки на веса, но не сводила числа. Это первое, что стоит прочитать в следующей сессии.

- **Кириллица у Paddle: уточнение.** В основной документации перечислены 5 типов текста, кириллицы

среди них нет, **но** Paddle отдельно публикует `cyrillic_PP-OCRv5_mobile_rec`. Утверждение «109 языков» из предыдущей выжимки **не подтверждено** — не повторяю его. Практический вывод: кириллическую ветку надо начинать не с нуля, а с этой готовой модели.

- **Числа SVTRv2, STR-Fewer-Labels, SynthText и прочих работ из раздела F не выписаны** — ветки

нашли и прочитали эти источники, но не успели свести измеренные цифры. Ссылки даны, чтобы открыть их и дочитать.

- **EasyOCR / Tesseract / MMOCR / RapidOCR**: внутренности, обучающие данные и метрики здесь не

приводятся — это направление A (`reports/research/A_ocr_engines.md`, готовится).

- **Точные цифры VLM** (OCRBench, OmniDocBench по каждой модели) — не проверялись; есть только

качественная часть из статьи PP-OCRv5 (локализация, галлюцинации, цена).

- **Qwen2.5-VL 72B ≈ 866 на OCRBench** — цифра из предыдущей выжимки, лично не подтверждена.
- **«Сколько именно данных нужно, чтобы выучить букву»** — точной таблицы в литературе не нашёл;

есть косвенные числа PP-OCRv5 (200→1000 кластеров при 600 тыс. сэмплов) и наша же точка отсчёта.

- **VLM на нашем домене не мерили** — ни одной прогона DeepSeek-OCR/Qwen по нашим картинкам.
- **Сколько сэмплов нужно на «сладкую зону» уверенности 0.95–0.97** в нашем домене — неизвестно,

это наш эксперимент, а не готовое число.

Документ собран 2026-10-21 в сессии главного агента. Пять параллельных веток поиска были остановлены по требованию владельца, не успев дописать свои отчёты, — но их логи сохранились, и всё содержательное из них (раздел F: PP-OCRv6, языковые модели Paddle, SVTRv2, ассортимент корпусов, коммерческие сервисы, бенчмарки) внесено сюда со ссылками. Измерения внутри документа помечены ссылками на `reports/` — они наши, а не заимствованные. PDF этой версии: `00_SYNTHESIS.pdf`.